ML_Lab_regularization_MNIST_UEN

October 19, 2026

1 IML Lab: Regularization in Logistic Regression using MNIST

We intentionally use only 1,000 training images so the effect is easier to observe.

1.1 Activity 1 — Load and Explore MNIST

MNIST contains 28×28 grayscale images.

Therefore each image has:

[28 x 28 = 784]

input features.

[23]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import fetch_openml
from sklearn.model_selection import train_test_split, StratifiedKFold,␣
↪GridSearchCV
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report

mnist = fetch_openml('mnist_784', version=1, as_frame=False)

X = mnist.data.astype('float32')
y = mnist.target.astype(int)

print("X shape:", X.shape)
print("y shape:", y.shape)
print("Number of classes:", len(np.unique(y)))
```

```
X shape: (70000, 784)
y shape: (70000,)
Number of classes: 10
```

1.2 Activity 2 — Create a Small Training Set

Create an 80:20 train-test split and then use only **1,000 training samples**.

Use `stratify` to preserve the class proportions.

Question: Why can reducing the training-set size make the effect of regularization easier to observe?

[24]:
```
X_train_full, X_test, y_train_full, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

X_train, _, y_train, _ = train_test_split(
    X_train_full,
    y_train_full,
    train_size=1000,
    random_state=42,
    stratify=y_train_full
)

print("Training samples:", X_train.shape[0])
print("Test samples:", X_test.shape[0])
print("Number of features:", X_train.shape[1])
```

```
Training samples: 1000
Test samples: 14000
Number of features: 784
```

1.3 Activity 3 — Scale the Pixel Values

Pixel values range from 0 to 255. Scale them to 0–1:

[25]:
```
X_train_scaled = X_train / 255.0
X_test_scaled = X_test / 255.0

print("Minimum:", X_train_scaled.min())
print("Maximum:", X_train_scaled.max())
```

```
Minimum: 0.0
Maximum: 1.0
```

2 Activity 4 — Investigate Different Regularization Strengths

Try:

`C_values = [0.001, 0.01, 0.1, 1, 10, 100]`

For every value record: - training accuracy, - test accuracy, - L2 norm of the coefficient matrix.

[26]:
```
models = {
    "C=0.001": LogisticRegression(C=0.001, max_iter=1000),
    "C=0.01": LogisticRegression(C=0.01, max_iter=1000),
    "C=0.1": LogisticRegression(C=0.1, max_iter=1000),
    "C=1": LogisticRegression(C=1, max_iter=1000),
    "C=10": LogisticRegression(C=10, max_iter=1000),
    "C=100": LogisticRegression(C=100, max_iter=1000)
}
```

[27]:
```
results = []

for name, model in models.items():
    model.fit(X_train_scaled, y_train)

    train_pred = model.predict(X_train_scaled)
    test_pred = model.predict(X_test_scaled)

    results.append({
        'Model': name,
        'Train Accuracy': accuracy_score(y_train, train_pred),
        'Test Accuracy': accuracy_score(y_test, test_pred),
        'Coefficient L2 Norm': np.linalg.norm(model.coef_)
    })

results_df = pd.DataFrame(results)
results_df.round(4)
```

[27]:

	Model	Train Accuracy	Test Accuracy	Coefficient L2 Norm
0	C=0.001	0.789	0.7562	0.784600
1	C=0.01	0.892	0.8424	2.626700
2	C=0.1	0.968	0.8670	6.026800
3	C=1	1.000	0.8626	12.504900
4	C=10	1.000	0.8579	20.525801
5	C=100	1.000	0.8496	38.261398

2.1 Activity 5 — Select `C` using Cross-Validation

Do not select `C` using the test set.

Use 5-fold stratified cross-validation on the training data.

Try:

`C_values = [0.001, 0.01, 0.1, 1, 10, 100]`

Then: 1. identify the best `C`; 2. train the final model; 3. evaluate it once on the test set.

[30]:
```
cv = StratifiedKFold(
    n_splits=5,
    shuffle=True,
```

```
    random_state=42
)

grid = GridSearchCV(
    LogisticRegression(
        max_iter=1000,
        random_state=42
    ),
    param_grid={'C': C_values},
    cv=cv,
    scoring='accuracy'
)

grid.fit(X_train_scaled, y_train)

print("Best C:", grid.best_params_['C'])
print("Best mean CV accuracy:", round(grid.best_score_, 4))
```

```
Best C: 0.1
Best mean CV accuracy: 0.875
```

[31]:
```
best_model = grid.best_estimator_
y_test_pred = best_model.predict(X_test_scaled)

print("Test accuracy:",
      round(accuracy_score(y_test, y_test_pred), 4))

print("\nClassification Report:")
print(classification_report(y_test, y_test_pred))
```

```
Test accuracy: 0.867

Classification Report:
              precision    recall  f1-score   support

           0       0.93      0.96      0.94      1381
           1       0.89      0.95      0.92      1575
           2       0.88      0.82      0.85      1398
           3       0.86      0.82      0.84      1428
           4       0.84      0.89      0.86      1365
           5       0.80      0.78      0.79      1263
           6       0.90      0.92      0.91      1375
           7       0.89      0.89      0.89      1459
           8       0.85      0.79      0.82      1365
           9       0.82      0.83      0.82      1391

    accuracy                           0.87     14000
   macro avg       0.87      0.87      0.86     14000
```

```
weighted avg       0.87      0.87      0.87     14000
```

Task 1 — Ridge-Regularized Logistic Regression Objective: Investigate how L2 regularization behaves when the amount of training data changes.

Students should: 1. Use the MNIST dataset. 2. Keep the test set fixed. 3. Create training subsets of: - 500 - 1,000 - 2,000 - 5,000 samples 4. Train Logistic Regression with L2 regularization. 5. Use a fixed value such as: C = 0.1

6. For each training size, calculate: Training Accuracy Test Accuracy Coefficient L2 Norm
7. Plot: Training Samples vs Training Accuracy Training Samples vs Test Accuracy Training Samples vs Coefficient L2 Norm

Task 2 — Elastic Net-Regularized Logistic Regression Use exactly the same experimental setup.

Students should: 1. Use the same MNIST dataset. 2. Use the same training sizes: - 500 - 1,000 - 2,000 - 5,000 3. Keep the test set unchanged. 4. Train Logistic Regression with Elastic Net:

[ ]: